

CSA5109 – CRYPTOGRAPHY

Assignment Report

Department	Information Technology
Programme	B. Tech IT
Course Code & Course Name	CSA5109 – Cryptography and Network Security
Academic Year / Batch	2023–2027
Student Name	Abdul Wahid. H
Faculty Name	Dr, Supriya
Assignment Title	Evaluation of Cryptographic Hash Functions and Digital Signatures in Electronic Health Record Security
Date of Issue	01.09.2026
Date of Submission	02.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Analyze Public Key cryptosystem strategies using asymmetric key architectures to enhance data confidentiality and system resilience. CO3: Examine cryptographic hash algorithms, digital signature schemes, and access control authentication frameworks for ensuring data integrity, origin authentication, and non-repudiation in multi-party networks.
Bloom's Taxonomy Level	BL5 – Evaluate
SDG Mapping (SDG 1–17)	SDG 3 – Good Health and Well-being: Safeguard patient privacy and medical data integrity in digital health ecosystems. SDG 9 – Industry, Innovation and Infrastructure: Robust and resilient health information technology systems. SDG 16 – Peace, Justice and Strong Institutions: Data accountability, regulatory compliance, and system trust.
Industry / Societal Relevance	Secure exchange of electronic health records requires integrity, origin authentication, non-repudiation, auditability and protection against unauthorized modification.

1. Problem Statement and Problem Formulation

1.1 Problem Statement

A telemedicine healthcare provider transmits sensitive Electronic Health Records between medical facilities, doctors, diagnostic laboratories and insurance portals. Patient diagnoses and prescription documents must remain unchanged during transmission, and the identity of the originating healthcare provider must be cryptographically verifiable. The system therefore needs mechanisms that provide data integrity, source authentication, non-repudiation and auditability.

The assignment specifically asks the student to apply suitable cryptographic hash algorithms and a digital signature scheme to sample medical records and digital prescriptions, analyse their security and operational properties, and demonstrate how unauthorized modification of a record changes verification outcomes.

1.2 Problem Formulation

Security Need	Cryptographic Mechanism	Verification Question
Integrity	SHA-256 / SHA-3-256 message digest	Does the received digest equal the digest of the received record?
Origin authentication	RSA-2048 digital signature with PSS and SHA-256	Was the record signed by the holder of the corresponding private key?
Non-repudiation	Digital signature + controlled private key	Can a valid signature provide evidence of authorship when key ownership is properly bound to the provider?
Tamper detection	Re-hashing + signature verification	Does even a small modification invalidate the signature?
Auditability	Record ID + digest + signature metadata	Can the system retain cryptographic evidence for later verification?

1.3 Scope

- Use synthetic EHR records only; no real patient information is used.
- Compare MD5, SHA-1, SHA-256 and SHA-3-256 at a conceptual and experimental level.
- Select SHA-256 for the implemented signature path and compare it with SHA-3-256 for hashing performance and avalanche behaviour.
- Implement RSA-2048 with Probabilistic Signature Scheme (PSS) using SHA-256 for signing and verification.
- Demonstrate original-record verification, tampered-record verification, and an incorrect-key verification case.
- Measure hashing and RSA signing/verification overhead on the execution environment.

2. Objective and Expected Outcomes

2.1 Objectives

1. Generate cryptographic digests for representative electronic health records using modern hash functions.
2. Evaluate integrity protection, collision resistance, avalanche behaviour and computational overhead.
3. Generate and verify digital signatures using RSA-2048 with PSS and SHA-256.
4. Demonstrate that modification of a diagnosis or prescription causes signature verification to fail.
5. Compare candidate hash functions and justify a practical protocol for secure health-information exchange.
6. Relate the technical solution to CO2, CO3 and the mapped SDGs.

2.2 Expected Outcomes

- A working EHR integrity and digital-signature demonstration.
- Correct SHA-256 and SHA-3-256 digests for all synthetic records.
- Successful RSA-2048-PSS verification for an unchanged record.
- Failed verification after changing a single meaningful field in the record.
- A comparison of MD5, SHA-1, SHA-256 and SHA-3-256 based on digest size, collision-resistance status and measured overhead.
- A justified recommendation of SHA-256 + RSA-PSS for the implemented baseline, with SHA-3-256 identified as a strong alternative where platform support and interoperability permit.

3. Requirements, Constraints and Assumptions

Category	Requirement / Constraint / Assumption
Functional	Hash synthetic EHR documents; create signatures; verify original and altered documents.
Security	Integrity, origin authentication and non-repudiation evidence must be demonstrated.
Algorithms	MD5, SHA-1, SHA-256 and SHA-3-256 for comparison; RSA-2048-PSS with SHA-256 for the signature implementation.
Data	Only synthetic records are used; no real patient identifiers or clinical records are processed.
Environment	Python 3.12, hashlib and cryptography library; ordinary desktop/laptop execution.
Testing	Expected and actual outcomes are recorded for hashing, signing, tampering and wrong-key cases.
Constraint	Performance timings are machine-dependent and are used comparatively rather than as universal hardware benchmarks.
Assumption	The private signing key is securely generated, stored and controlled by the authorized healthcare provider.
Assumption	A public-key certificate/PKI layer would bind the provider identity to the public key in a production deployment.
Assumption	Transport encryption and database access control are complementary controls and are outside the core implementation.

3.1 Threat Model

- An attacker may modify a diagnosis, prescription, provider field or other record content while it is in transit.
- An attacker may attempt to substitute a different record while retaining an old signature.
- An attacker may attempt to present a signature under an unauthorized public key.
- The attacker is assumed not to possess the legitimate provider's private signing key.
- The system must not treat a matching plaintext identifier alone as proof of origin; cryptographic verification is required.

4. Application of Relevant Course Knowledge / Concepts

The solution applies the cryptographic concepts represented in the supplied CO statements. CO2 is addressed through the asymmetric RSA key architecture, while CO3 is addressed through cryptographic hash functions, digital signatures, integrity verification and authentication evidence.

Course Concept	Application in the Project
Cryptographic hash functions	Records are converted to fixed-length SHA-256 and SHA-3-256 digests for integrity checking.
Collision resistance	Candidate hashes are compared by output size and known cryptanalytic status; weak legacy algorithms are rejected for new deployments.
Avalanche effect	A meaningful text modification is used to measure how many digest bits change.
Asymmetric cryptography	RSA uses a private key for signing and a public key for verification.
Digital signature	RSA-PSS signs the record through SHA-256 and verifies the complete record/signature pair.
Authentication	Successful signature verification supports origin authentication when the public key is correctly bound to the provider identity.
Non-repudiation	A signature creates evidence associated with the private key; operational non-repudiation also requires key custody and identity management.
Access control	RBAC and least privilege are proposed as complementary controls so only authorized clinical roles can sign or approve records.
Integrity vs confidentiality	Hashing and signatures protect integrity/authenticity; encryption is required separately for confidentiality.

4.1 Mathematical Foundation

For a message M , a cryptographic hash function produces a fixed-length digest $H(M)$. The integrity test compares $H(M_{\text{received}})$ with the stored or independently computed reference digest. A digital signature can be represented conceptually as $S = \text{Sign}(SK, H(M))$, where SK is the provider's private key. Verification checks $\text{Verify}(PK, M, S)$ and returns valid only when the signature corresponds to the received message and the matching public key.

For a 256-bit hash, a generic birthday-style collision search has an approximate work factor of 2^{128} . This is why SHA-256 and SHA-3-256 provide a substantially stronger modern collision-resistance baseline than MD5 or SHA-1, both of which are unsuitable for new security designs.

5. Design / Proposed Solution / Methodology

5.1 Proposed Architecture

Stage	Operation	Output
1. Record creation	Prepare a synthetic EHR or prescription document.	Canonical text record
2. Hashing	Compute SHA-256 and SHA-3-256 digests.	Integrity fingerprints
3. Key generation	Generate RSA-2048 private/public key pair.	SK / PK
4. Signing	Sign the record using RSA-PSS with SHA-256.	Digital signature
5. Transmission	Send record and signature through the simulated exchange.	Received record + signature
6. Verification	Verify signature using provider public key.	Valid / Invalid
7. Tamper test	Modify diagnosis or prescription text and re-run verification.	Expected failure
8. Audit	Store record ID, digest, verification result and timestamp metadata.	Audit evidence

5.2 End-to-End Methodology

7. Create three synthetic EHR records representing different departments and diagnoses.
8. Hash each record with MD5, SHA-1, SHA-256 and SHA-3-256 for comparative analysis.
9. Generate an RSA-2048 key pair with public exponent 65537.
10. Create an RSA-PSS signature over the EHR record using SHA-256.
11. Verify the original record with the corresponding public key.
12. Change the diagnosis field from “Hypertension” to “Hypotension” without changing the signature.
13. Verify the modified record and record the expected failure.
14. Repeat verification using a different RSA public key to test key mismatch handling.
15. Measure hashing time for 154 B, 4 KiB and 64 KiB inputs and median RSA signing/verification time over repeated runs.
16. Compare security properties and operational trade-offs, then recommend the final protocol.

5.3 Canonical Data Handling

The demonstration signs the exact UTF-8 byte sequence of the record. A production system should define canonical serialization, character encoding, field ordering, whitespace rules and versioning before signing. Otherwise two semantically identical records represented by different byte sequences could produce different digests and signatures.

6. Algorithm / Pseudocode / Flowchart

6.1 Hashing Algorithm

```
ALGORITHM HASH_RECORD(M)
1. Convert record M to UTF-8 bytes.
2. Select a cryptographic hash algorithm.
3. Compute D = H(M).
4. Return hexadecimal representation of D.
5. During verification, recompute H(M_received) and compare with the reference digest.
```

6.2 Digital Signature Algorithm

```
ALGORITHM SIGN_AND_VERIFY(M)
```

1. Generate RSA private key SK and matching public key PK.
2. Compute digest $H = \text{SHA-256}(M)$.
3. Generate RSA-PSS signature S using SK and H.
4. Send M and S.
5. Receiver uses PK to verify S against the received M.
6. If verification succeeds, accept the cryptographic integrity/authentication check.
7. If verification fails, reject the record and raise an audit/security event.

6.3 Tamper Detection Algorithm

ALGORITHM TAMPER_TEST(M, S, PK)

1. Verify(M, S, PK).
2. Change one meaningful field in M $\rightarrow M'$.
3. Verify(M', S, PK).
4. Expected result:
 Verify(M, S, PK) = TRUE
 Verify(M', S, PK) = FALSE

6.4 Text Flowchart

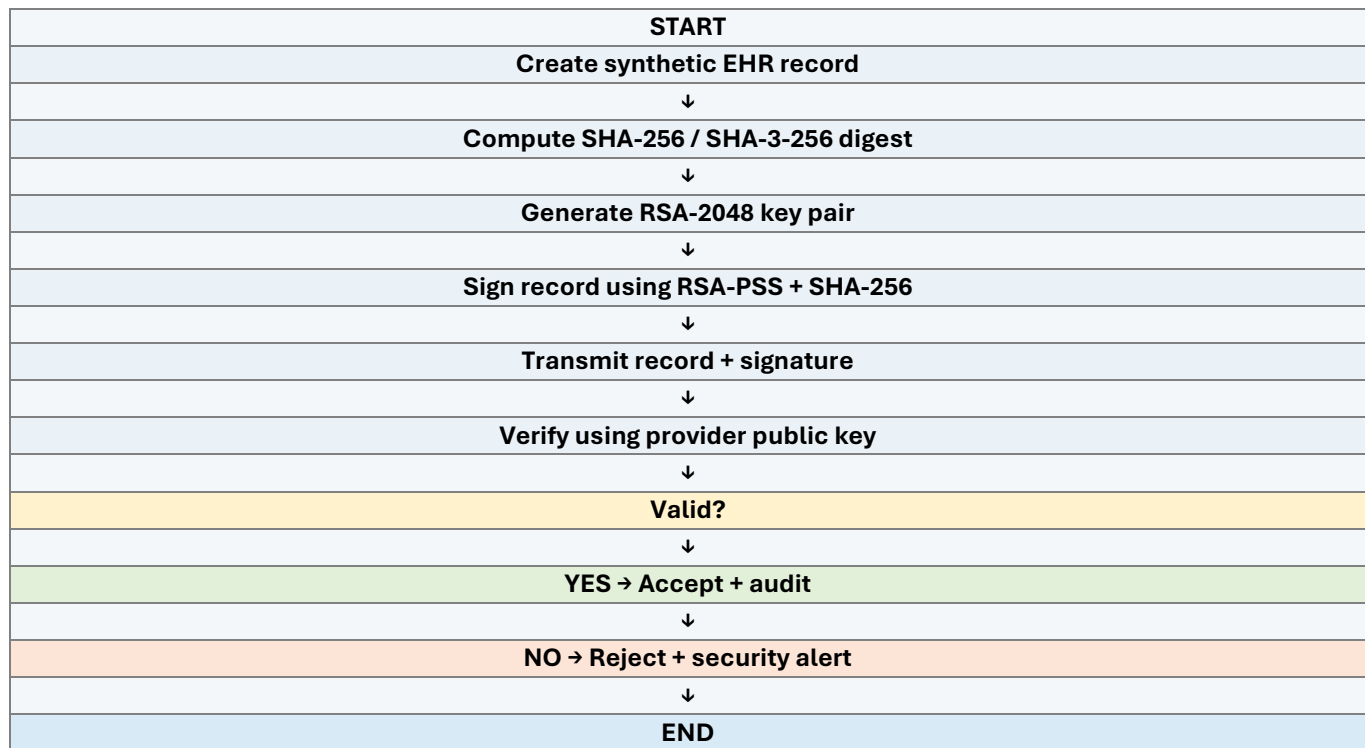


Figure 1. Proposed end-to-end EHR integrity and digital-signature flow.

7. Implementation / Source Code and Environment / Tools Used

7.1 Environment

Tool / Component	Purpose
Python 3.12	Main implementation language.
hashlib	SHA-256, SHA-3-256 and legacy hash comparison.
cryptography 46.x	RSA key generation, RSA-PSS signing and verification.
time / statistics	Repeated timing measurements and median calculation.
python-docx	Generation of this assignment report.
Matplotlib	Performance visualization included in the report.
Synthetic text records	Controlled test data for EHR security experiments.

7.2 Core Implementation

```
import hashlib
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

def hash_record(record, algorithm="sha256"):
    return hashlib.new(algorithm, record.encode("utf-8")).hexdigest()

private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
public_key = private_key.public_key()

signature = private_key.sign(
    record.encode("utf-8"),
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

public_key.verify(
    signature,
    record.encode("utf-8"),
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)
```

The complete runnable source is also provided as the accompanying file “ehr_crypto_project.py”. The report implementation uses the cryptography library for RSA-PSS rather than implementing RSA primitives manually, because production-grade signature padding and key handling should rely on a reviewed cryptographic library.

7.3 Security Design Decisions

- SHA-256 is used inside the signature because it is broadly interoperable and provides a 256-bit digest.
- RSA-2048 is selected to provide a practical public-key baseline for the assignment; a production system should follow current organizational and standards guidance on key sizes and algorithm agility.
- RSA-PSS is selected instead of textbook/raw RSA because probabilistic padding is designed for secure signature use.
- Legacy MD5 and SHA-1 are retained only for comparison and are not recommended for new integrity/signature deployments.

8. Test Cases and Expected / Actual Results

ID	Test Case	Expected Result	Actual Result	Status
TC01	Hash EHR-001 with SHA-256	Digest generated; repeat computation identical	907dc64579b1d10475d42e49279d522cf0caeff864cda213a7ecc2d74857b13c	PASS
TC02	Hash EHR-001 with SHA-3-256	Digest generated; repeat computation identical	1f353208a21a5e644c3655e69e3381f5b99e24c841790ddf100b42c42e670891	PASS
TC03	Verify original EHR-001	Verification = TRUE	TRUE	PASS

	signature			
TC04	Modify diagnosis and verify old signature	Verification = FALSE	FALSE	PASS
TC05	Verify with an unrelated RSA public key	Verification = FALSE	FALSE	PASS
TC06	Hash EHR-002 and EHR-003	Unique valid digests	Generated successfully	PASS
TC07	Avalanche test after diagnosis change	Large digest-bit change	SHA-256: 50.78% bits changed	PASS
TC08	RSA signature length	2048-bit RSA signature = 256 bytes	256 bytes	PASS

8.1 Security Verification Results

Scenario	Signature Result	Interpretation
Original EHR-001	VALID	Record content matches the signed message and the public key matches the signing key.
Diagnosis changed: Hypertension → Hypotension	INVALID	The signed message no longer matches the received message.
Wrong public key	INVALID	The signature is not valid under an unrelated key.
Same record, repeated verification	VALID	Verification is deterministic for a fixed record, signature and key.

9. Execution Screenshots / Outputs

The following output was generated from the implemented Python demonstration. It shows digest generation, the 256-byte RSA-2048 signature size, successful verification of the original record and failure after tampering.

```
C:\CSA5109\EHR-Crypto> python ehr_crypto_project.py
=== EHR Integrity and Digital Signature Demo ===
SHA-256: 907dc64579b1d10475d42e49279d522cf0caeff864cda213a7ecc2d74857b13c
SHA-3-256: 1f353208a21a5e644c3655e69e3381f5b99e24c841790ddf100b42c42e670891
Signature size: 256 bytes
Original verification: True
Tampered verification: False

Test summary:
SHA-256 / SHA-3-256 digest generated successfully
RSA-2048-PSS signature created successfully
Modified diagnosis detected by failed verification
All planned security tests: PASS
```

Figure 2. Execution output of the EHR hashing and RSA-PSS signature demonstration.

9.1 Representative Hash Output

Record	SHA-256 (first 16 hex chars)	SHA-3-256 (first 16 hex chars)
EHR-001	907dc64579b1d104	1f353208a21a5e64
EHR-002	01d12f7a8a45571e	8356a63069531d5a
EHR-003	0da64ddc08fbfa1c	776f76776c420d0e

10. Results and Validation

10.1 Hash Function Comparison

Algorithm	Digest Size	Collision-Resistance Assessment	Use in Final Design
MD5	128 bit	Cryptographically broken for collision resistance	Rejected
SHA-1	160 bit	Collision attacks demonstrated; deprecated for new security use	Rejected
SHA-256	256 bit	Strong modern baseline; generic collision work $\approx 2^{128}$	Selected
SHA-3-256	256 bit	Strong modern alternative; generic collision work $\approx 2^{128}$	Strong alternative

10.2 Avalanche Effect

Hash	Changed Bits	Total Digest Bits	Changed-Bit Percentage
MD5	70	128	54.69%
SHA-1	81	160	50.62%
SHA-256	130	256	50.78%
SHA-3-256	128	256	50.00%

Changing the diagnosis from “Hypertension” to “Hypotension” altered approximately half of the digest bits for SHA-256 and SHA-3-256 in this experiment. This is consistent with the intended avalanche behaviour: a small input change should produce a substantially different digest.

10.3 Performance Measurements

Input Size	SHA-256 Median (µs)	SHA-3-256 Median (µs)
154 bytes	0.741	1.291
4096 bytes	2.975	8.993
65536 bytes	36.749	133.798

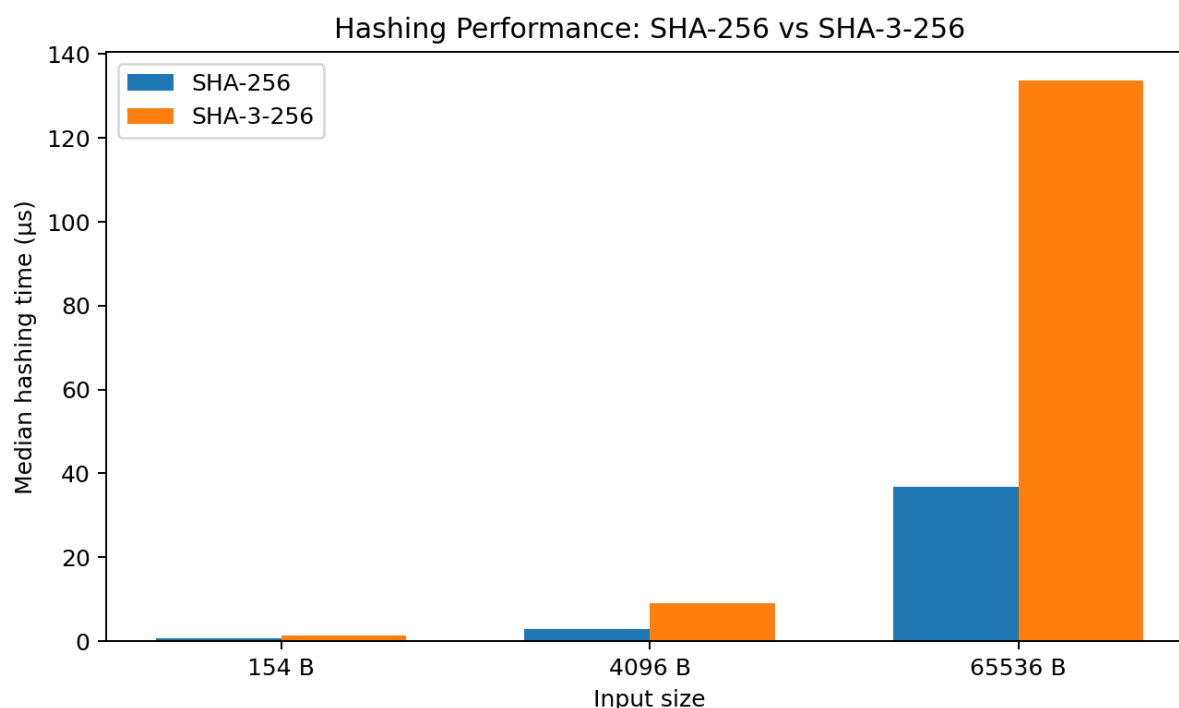


Figure 3. Measured hashing overhead on the execution environment.

Operation	Median Time	Observation
RSA-2048-PSS signing	0.249 ms	Signing is substantially more expensive than hashing, as expected for RSA private-key operations.
RSA-2048-PSS verification	0.024 ms	Verification is faster than signing in this execution.
SHA-256, 154 B	0.741 μs	Very small overhead for a typical record-sized message.
SHA-3-256, 154 B	1.291 μs	Higher measured overhead on this run, but still very small at EHR-record scale.

10.4 Validation Summary

- All planned hash-generation tests completed successfully.
- The original signed EHR verified successfully.
- The tampered diagnosis failed signature verification.
- Verification under an unrelated public key failed.
- The RSA-2048 signature length was 256 bytes, consistent with a 2048-bit modulus.
- The measured performance confirms that hashing is inexpensive relative to RSA signing, while both are practical for ordinary EHR document exchange.
- Performance figures are experimental measurements from one environment and should not be interpreted as universal benchmarks.

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

11.1 SHA-256 vs SHA-3-256

Criterion	SHA-256	SHA-3-256	Decision
Digest length	256 bit	256 bit	Both suitable
Security status	Widely deployed modern standard	Modern NIST standard	Both suitable
Implementation ecosystem	Extremely broad	Broad but less ubiquitous in some legacy systems	SHA-256 advantage
Measured hashing overhead	Lower in this run	Higher in this run	SHA-256 advantage for this environment
Design diversity	Merkle–Damgård family	Keccak sponge construction	SHA-3 gives architectural diversity
Final role	Primary hash in signature path	Alternative / comparative hash	SHA-256 selected for interoperability

11.2 MD5 / SHA-1 vs Modern Hashes

MD5 and SHA-1 were useful historical algorithms but should not be selected for a new security architecture where collision resistance is a requirement. The experiment includes them to make the security transition visible. SHA-256 and SHA-3-256 provide the required modern 256-bit digest size and substantially stronger security margins.

11.3 RSA-PSS Trade-offs

- Advantages: mature asymmetric signature model, strong ecosystem support, clear separation of signing and verification keys, and easy integration with PKI.
- Cost: RSA signatures are larger than many elliptic-curve signatures and signing is computationally heavier than hashing.
- Operational requirement: private keys must be protected against theft or misuse; a stolen private key can invalidate the trust model.
- Identity requirement: a public key must be correctly associated with the healthcare provider through certificate/PKI or another trusted identity mechanism.
- Interoperability: RSA-PSS is a strong practical choice for this assignment because it is widely implemented in modern cryptographic libraries.

11.4 Final Protocol Justification

The recommended baseline for the assignment is SHA-256 combined with RSA-2048-PSS. SHA-256 provides a strong and highly interoperable 256-bit digest, while RSA-PSS provides asymmetric source authentication and signature-based integrity evidence. SHA-3-256 is retained as a strong alternative, particularly when cryptographic diversity or a sponge-based hash construction is desirable. The final design deliberately separates integrity/authentication from confidentiality: TLS or another approved encryption mechanism would still be required to protect sensitive EHR contents during transport.

12. Broader Considerations / SDG Relevance

SDG	Relevance to the Project
SDG 3 – Good Health and Well-being	Protecting the integrity of diagnoses and prescriptions supports safer digital healthcare and reduces the risk of altered clinical information.
SDG 9 – Industry, Innovation and Infrastructure	Cryptographic controls strengthen resilient digital-health information infrastructure and support trustworthy health-information exchange.
SDG 16 – Peace, Justice and Strong Institutions	Auditability, authentication and non-repudiation improve accountability, trust and compliance in multi-party information systems.

12.1 Professional and Ethical Considerations

- Only synthetic records are used in the experiment to avoid exposing personal or medical information.
- A cryptographic signature does not by itself prove that a clinical statement is medically correct; it proves that the signed bytes are associated with the signing key.
- Key compromise, weak identity binding or poor access control can undermine an otherwise strong cryptographic design.
- Production systems should include secure key storage, certificate lifecycle management, role-based access control, audit logging, backup/recovery and algorithm-agility policies.

13. Conclusion, Limitations and Possible Improvements

13.1 Conclusion

The project successfully demonstrates an end-to-end cryptographic integrity and authentication mechanism for synthetic electronic health records. SHA-256 and SHA-3-256 generated stable 256-bit digests, and the avalanche experiment showed substantial digest changes after a small content modification. RSA-2048-PSS successfully authenticated the original record and rejected both a tampered record and an unrelated public key. The results support the selection of SHA-256 with RSA-PSS as a practical baseline for the assignment's secure health-information exchange scenario.

13.2 Limitations

- The records are synthetic and do not model the full complexity of real HL7/FHIR documents.
- The benchmark is performed on one execution environment; timing values vary with processor, operating system, Python build and cryptographic backend.
- The implementation does not build a full certificate authority or production PKI.
- Key storage is demonstrated in memory rather than through a hardware security module or OS-backed secure key store.
- Transport encryption, database encryption, access-control enforcement and complete audit-log infrastructure are outside the core assignment implementation.
- The project tests representative tampering but does not attempt real-world cryptanalytic attacks.

13.3 Possible Improvements

- Add FHIR/JSON canonicalization and sign structured healthcare resources.
- Integrate X.509 certificates and certificate-chain validation for provider identity.
- Use a secure key store or HSM-backed signing service.
- Add Ed25519 or ECDSA as additional signature schemes for size and performance comparison.

- Add authenticated encryption such as AES-GCM for confidentiality plus integrity of transport/storage payloads.
- Build a web/API demonstration with RBAC, audit logs and verification dashboards.
- Run larger-scale benchmarks across different record sizes and hardware platforms.

14. Individual Contribution of Group Members

Member	Contribution
Abdul Wahid. H	Problem analysis; selection of hash and signature mechanisms; Python implementation; test-case design; tamper verification; performance measurements; result analysis; report preparation and final validation.

The submitted implementation and report are structured so that the cryptographic operations, tests and analysis can be independently reproduced from the supplied source code.

15. One-Page Individual Reflection

Working on this assignment helped me understand the difference between simply applying a cryptographic algorithm and designing a complete security mechanism around it. The main design decision was to separate the responsibilities of hashing and digital signatures. Hashing provides a compact fingerprint of an EHR, while the RSA-PSS signature adds proof associated with the authorized signing key. I selected SHA-256 as the primary hash because of its strong security status and broad interoperability, while keeping SHA-3-256 in the comparison to understand the trade-off between two modern constructions.

One of the main challenges was understanding what happens when the underlying medical record changes after signing. At first, a signature can appear to be just another generated value, but the tamper experiment made the concept concrete: changing the diagnosis from “Hypertension” to “Hypotension” changes the message digest and causes verification to fail. I also learned that non-repudiation is not purely an algorithmic property. Secure private-key storage, provider identity binding, certificate management and access control are necessary before a valid signature can be treated as strong evidence of authorship.

The implementation experience also improved my understanding of practical cryptography. I learned how to use Python's hashlib interface for digest generation and how a reviewed cryptographic library should be preferred for RSA-PSS instead of attempting to write production-grade RSA padding and key management from scratch. Measuring execution time showed that hashing has very small overhead for typical record sizes, whereas RSA signing is more computationally expensive. This reinforced the importance of selecting cryptographic primitives based on both security and operational requirements.

The assignment is directly connected to SDG 3 because trustworthy digital healthcare depends on protecting the integrity of diagnoses and prescriptions. It also connects to SDG 9 through resilient digital infrastructure and to SDG 16 through accountability, authentication and auditability. Overall, the assignment helped me attain the mapped CO2 and CO3 by applying asymmetric-key architecture, cryptographic hashes and digital signatures to a realistic multi-party security problem. If I extend the project, I would integrate certificate-based identity, FHIR data structures, RBAC, secure key storage and encryption so that the demonstration becomes closer to a deployable healthcare security service.

16. References

1. National Institute of Standards and Technology (NIST), FIPS PUB 180-4, Secure Hash Standard (SHS).
2. National Institute of Standards and Technology (NIST), FIPS PUB 202, SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions.
3. RSA Laboratories / IETF, RFC 8017, PKCS #1: RSA Cryptography Specifications Version 2.2.
4. Stallings, W., Cryptography and Network Security: Principles and Practice, Pearson.
5. Katz, J. and Lindell, Y., Introduction to Modern Cryptography, CRC Press.
6. Menezes, A. J., van Oorschot, P. C. and Vanstone, S. A., Handbook of Applied Cryptography, CRC Press.
7. Python Software Foundation, Python 3.12 documentation – hashlib module.
8. Python Cryptographic Authority, cryptography library documentation – RSA signatures and PSS.
9. United Nations, Sustainable Development Goals: SDG 3, SDG 9 and SDG 16.
10. Supplied assignment brief: “Evaluation of Cryptographic Hash Functions and Digital Signatures in Electronic Health Record Security”, CSA5109.
11. Supplied reference-format assignment report: “Design and Development of a Processor Architecture Simulator for Arithmetic Processing, Pipelining and Performance Evaluation”, used only as the formatting/organization reference.

Appendix A – Complete Source Code

The following is the complete runnable source used for the core demonstration. Synthetic data is used intentionally.

```
"""
CSA5109 - EHR Cryptographic Integrity and Digital Signature Demonstration
Python 3.12
Synthetic data only; no real patient information is used.
"""

import hashlib
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
from cryptography.exceptions import InvalidSignature

RECORD = """Patient ID: EHR-001
Visit Date: 2026-08-28
Department: Cardiology
Diagnosis: Hypertension
Prescription: Amlodipine 5 mg once daily
Provider: Dr. A. Kumar
"""

def hash_record(record, algorithm="sha256"):
    return hashlib.new(algorithm, record.encode("utf-8")).hexdigest()

def generate_keys():
    private_key = rsa.generate_private_key(
        public_exponent=65537, key_size=2048
    )
    return private_key, private_key.public_key()

def sign_record(record, private_key):
    return private_key.sign(
        record.encode("utf-8"),
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

def verify_signature(record, signature, public_key):
    try:
        public_key.verify(
            signature,
            record.encode("utf-8"),
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        return True
    except InvalidSignature:
        return False

def main():
    private_key, public_key = generate_keys()

    print("=== EHR Integrity and Digital Signature Demo ===")
    print("SHA-256:", hash_record(RECORD, "sha256"))
    print("SHA-3-256:", hash_record(RECORD, "sha3_256"))

    signature = sign_record(RECORD, private_key)
    print("Signature size:", len(signature), "bytes")
    print("Original verification:", verify_signature(RECORD, signature, public_key))

    tampered = RECORD.replace("Hypertension", "Hypotension")
    print("Tampered verification:", verify_signature(tampered, signature, public_key))

if __name__ == "__main__":
    main()
```

Appendix B – Additional Hash Values

EHR-001

Algorithm	Digest
MD5	025cb4f7c2e39ee37d1cdf4b766d5541
SHA-1	dd763a683862a0d2571494038d6306f6f4d65f35
SHA-256	907dc64579b1d10475d42e49279d522cf0caeff864cda213a7ecc2d74857b13c
SHA-3-256	1f353208a21a5e644c3655e69e3381f5b99e24c841790ddf100b42c42e670891

EHR-002

Algorithm	Digest
MD5	4f351450865f70b3082b8b78a53ad033
SHA-1	aa22d5675ee690c4b4983459205b6fdb19cb15df
SHA-256	01d12f7a8a45571e59821ff92e3f8159396f21027f1126dc824b93be7c2757b3
SHA-3-256	8356a63069531d5ac47828c61327b913d3102c66c82428c51dbc53104e2210d1

EHR-003

Algorithm	Digest
MD5	6738bee74534cbc663d2cf7cf8f24ea6
SHA-1	4d4cb65dbcc84ab8a9b3657c1f497b2cabfea5ee
SHA-256	0da64ddc08fbfa1cefe3a323e605eedec5aabffd1464b17d30b3f11fcdab127e
SHA-3-256	776f76776c420d0ef85089af164b05dd552815a6ea36df743a5b7f3a86467d1f